

RESTAURANTOS MASTER SPECIFICATION V2.0

PART 1 — PRODUCT FOUNDATION, DESIGN SYSTEM, GLOBAL RULES & ARCHITECTURE

1. PROJECT IDENTITY

RestaurantOS is not:

A restaurant website template

A restaurant dashboard

A website builder

A CMS

RestaurantOS is:

> A multi-tenant Restaurant Operating System allowing restaurants to launch, manage, market and operate their entire business from one platform.

The platform must feel closer to:

Shopify

Stripe Dashboard

Webflow

Framer

Notion

Hubspot

than it feels to:

WordPress templates

Generic admin dashboards

ThemeForest products

2. PRIMARY OBJECTIVE

Every restaurant that purchases RestaurantOS should be able to:

Launch

Website

Menu

Reservations

Online Ordering

Manage

Content

Branding

Media

Menu

Customers

Grow

Marketing

SEO

Analytics

Promotions

without requiring a developer.

3. NON NEGOTIABLE RULES

These rules override everything.

RULE 1

NO HARD-CODED RESTAURANT DATA

Never hardcode:

restaurant names

logos

phone numbers

addresses

opening hours

menu items

hero images

Everything must come from:

RestaurantData

RULE 2

NO HARD-CODED IMAGES

Never ship:

steaks

wine glasses

burgers

pasta

as permanent assets.

Every image must be:

Upload Image

Replace Image

Delete Image

Media Library

RULE 3

NO DUPLICATED SETTINGS

Bad:

Restaurant Name

appears in

Profile

AND

Website

AND

Theme

AND

Footer

Good:

Restaurant Name

exists once.

Everything else references it.

RULE 4

EVERY CHANGEABLE THING MUST HAVE:

Save Draft

Publish

Undo

Redo

Autosave

RULE 5

NO PAGE SHOULD FEEL FINISHED AFTER ONE SCREEN

Every major module must support expansion.

Future-proof architecture.

4. DESIGN PHILOSOPHY

The current website is too tied to the mockup.

The mockup is:

Inspiration

NOT

Implementation

REMOVE:

Mac browser frames

Fake desktop UI

Decorative sparkles

Decorative glow particles

Placeholder labels

These are presentation artifacts.

KEEP:

Premium feel

Spacious layouts

Luxury aesthetic

Strong typography

Large imagery

Elegant spacing

5. DESIGN LANGUAGE

RestaurantOS must feel:

Premium

Confident

Minimal

Professional

Fast

NOT

Fancy

Flashy

Overdesigned

6. SPACING SYSTEM

Use ONLY:

4

8

12

16

24

32

48

64

96

No random spacing.

7. BORDER RADIUS

Use ONLY:

8px

12px

16px

24px

8. SHADOWS

Three shadows only.

Small

Inputs

Cards

Medium

Dropdowns

Modals

Large

Dialogs

Inspector panels

No other shadow systems.

9. TYPOGRAPHY

Maximum:

1 Display Font

Headlines

1 Body Font

Everything else

Never introduce third fonts.

10. COLOR SYSTEM

DO NOT BUILD A GOLD WEBSITE

The gold aesthetic belongs to:

Demo Theme

Only

Platform architecture:

Theme

Brand

Content

must be separated.

Every restaurant can have:

Black + Gold

White + Green

Red + Cream

Blue + Silver

without code changes.

11. CUSTOMER WEBSITE ARCHITECTURE

Current pages:

Home

Menu

Reservations

Services

Locate Us

Connect

Checkout

Good.

But every page must be generated from:

RestaurantData

Never page-specific hardcoded content.

12. WEBSITE EDITOR PRINCIPLE

Website editing should feel like:

Shopify

+

Webflow

+

Framer

NOT

WordPress settings page

13. WEBSITE MANAGER

New structure:

Website

|

├── Pages

├── Navigation

└─ Homepage
└─ Footer
└─ SEO
└─ Forms
└─ Theme
└─ Publish

Current architecture is fragmented.

Unify it.

14. RCC RENAMING

Do not use:

RCC

in UI.

Internal only.

Use:

Website Manager

or

RestaurantOS Workspace

15. SIDEBAR STRUCTURE

Required:

Dashboard

Website

- Pages

- Navigation

- Theme

- SEO

Restaurant

- Profile

- Locations

- Hours

Menu

- Categories

- Items

- Specials

Orders

Reservations

Customers

Media Library

Marketing

Analytics

Settings

Support

16. SIDEBAR BEHAVIOUR

Desktop:

Expanded

Tablet:

Collapsible

Mobile:

Drawer

Always show:

active page

active section

17. DASHBOARD

Dashboard must answer:

What happened?

What is happening?

What needs attention?

Immediately.

Required cards:

Orders Today

Reservations Today

Revenue Today

New Customers

Required widgets:

Recent Orders

Upcoming Reservations

Popular Items

Website Traffic

18. GLOBAL SEARCH

Required.

Search:

Menu Items

Customers

Orders

Reservations

Pages

One search box.

19. GLOBAL NOTIFICATIONS

Required.

For:

new order

reservation

failed payment

publish success

20. MEDIA LIBRARY

One centralized media system.

Never upload separately.

Everything uses:

Media Library

Media supports:

Upload

Replace

Delete

Rename

Search

Filter

Grid

List

Preview

21. IMAGE RULES

Every image supports:

Alt Text

Caption

Crop

Compression

Usage Count

22. BUTTON HIERARCHY

Primary

One per section.

Secondary

Multiple allowed.

Danger

Delete only.

Ghost

Low priority.

Never use 5 different button styles.

23. FORM RULES

Every form:

Required states:

Default

Focus

Success

Error

Disabled

Loading

24. SAVE SYSTEM

Every editable page:

Must show:

Unsaved Changes

Saving...

Saved

Published

Draft

25. RESPONSIVENESS

Must support:

Mobile

Tablet

Desktop

Large Desktop

No desktop-first hacks.

26. ACCESSIBILITY

Required:

Keyboard navigation

Focus states

ARIA labels

Alt text

Color contrast

27. PERFORMANCE

Target:

Lighthouse 90+

Minimum.

28. REUSABLE COMPONENTS

Create:

PageHeader

DataTable

EmptyState

UploadField

SaveBar

InspectorPanel

StatusBadge

MetricCard

SectionCard

Reuse everywhere.

Never duplicate.

29. FUTURE FIREBASE RULE

Everything must already be structured for:

RestaurantData

even before Firebase exists.

No rewrites later.

30. EXECUTION ORDER

Before adding new features:

1. Audit current codebase

2. Audit current admin

3. Audit current customer website

4. Compare against this specification

5. Identify inconsistencies

6. Refactor architecture

7. Standardize design system

8. Standardize navigation

9. Standardize components

10. THEN continue feature development

FINAL INSTRUCTION TO REPLIT

You are not continuing a prototype.

You are transforming RestaurantOS into a production-ready SaaS platform.

Before writing any code:

Audit everything.

Document everything.

Identify every inconsistency.

Refactor before expanding.

Preserve working functionality.

Remove technical debt.

Unify architecture.

The goal is not to generate pages.

The goal is to build a coherent Restaurant Operating System.

--